PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

ALGORITMOS AVANZADOS

Primer Examen
(Segundo Semestre 2026)

Duración: 2h 50 min.

- **No puede utilizar apuntes, solo hojas sueltas en blanco.**
- En cada función el alumno deberá incluir, a modo de comentario, la forma de solución que utiliza para resolver el problema. De no incluirse dicho comentario, el alumno perderá el derecho a reclamo en esa pregunta.
- No puede emplear plantillas o funciones no vistas en los cursos de programación de la especialidad.
- Los programas deben ser desarrollados en el lenguaje C++. Si la implementación es diferente a la estrategia indicada o no la incluye, la pregunta no será corregida.
- Un programa que no muestre resultados coherentes y/o útiles será corregido sobre el 50% del puntaje asignado a dicha pregunta.
- Debe utilizar comentarios para explicar la lógica seguida en el programa elaborado. El orden será parte de la evaluación.
- Se utilizarán herramientas para la detección de plagios, por tal motivo si se encuentran soluciones similares, se anulará la evaluación a todos los implicados y se procederá con las medidas disciplinarias dispuestas por la FCI.
- **Solo está permitido acceder a la plataforma de PAIDEIA, cualquier tipo de navegación, búsqueda o uso de herramientas de comunicación se considera plagio por tal motivo se anulará la evaluación y se procederá con las medidas disciplinarias dispuestas por la FCI.**
- Para esta evaluación solo se permite el uso de las librerías **iostream, iomanip, climits cmath, fstream, vector, map, iterator, algorithm o cstring. No se permite el uso de variables globales.**
- Su trabajo deberá ser subido a PAIDEIA.
- **Es obligatorio usar como IDE CLion.**
- Los proyectos deben llevar como nombre su código de la siguiente forma codigo_EX1_P# (donde # representa el número de la pregunta a resolver)

Seleccione 2 de las siguientes 3 preguntas:

Pregunta 1 (10 puntos)

Una organización debe distribuir N proyectos indivisibles entre tres equipos: A, B y C. Cada proyecto tiene un número positivo de horas de trabajo. Para que la distribución sea justa, los tres equipos deben recibir exactamente la misma carga total. Además, se conocen M pares de proyectos incompatibles. Dos proyectos incompatibles no pueden asignarse al mismo equipo, porque requieren herramientas que no pueden compartirse. Considere las siguientes variables y restricciones:

- N es el número de proyectos.
- M es el número de pares incompatibles.
- horas es un arreglo de N elementos; horas[i] representa las horas del proyecto i + 1 (1 <= horas[i] <= 100).
- incompatibilidades es un arreglo bidimensional de M filas y 2 columnas. Cada fila {u, v} indica que los proyectos u y v no pueden pertenecer al mismo equipo.
- Los proyectos se numeran desde 1 hasta N.
- No debe solicitar ni leer datos desde consola.
- Para probar otros casos, solo se deben modificar los valores de N, M, horas e incompatibilidades.
- Los datos se definen en el main mediante las siguientes constantes y arreglos.

- Los índices de proyectos pueden aparecer en cualquier orden dentro de cada equipo. Si existen varias soluciones, basta con mostrar una.
- Si se puede encontrar una solución, se imprimirá:

  Equipo A: <indices de proyectos>
  Equipo B: <indices de proyectos>
  Equipo C: <indices de proyectos>
  Carga por equipo: <carga>

A continuación, presentamos algunos ejemplos:

Caso 1:

Datos de entrada:

```
const int N = 6;
const int M = 2;
int horas[N] = {3, 5, 4, 4, 3, 2};
int incompatibilidades[M][2] = {{1, 2}, {3, 4}};
```

Salida

```
Equipo A: 1 3
Equipo B: 2 6
Equipo C: 4 5
Carga por equipo: 7
```

Caso 2:

Datos de entrada:

```
int horas[N] = {3, 5, 4, 4, 3, 2};
int incompatibilidades[M][2] = {{1, 3},{2, 6}};
```

Salida
Si no existe una distribución válida, se imprimirá:
IMPOSIBLE

Diseñe e implemente un algoritmo de backtracking que determine si existe una asignación válida. Si existe, debe mostrar una asignación; de lo contrario, debe indicar que es imposible.

Pregunta 2 (10 puntos)

La fábrica BTSArmy ha dividido su proceso de fabricación de robots en **N procesos consecutivos**, identificados desde el proceso 0 hasta el proceso N−1. Para ejecutar estos procesos, la fábrica dispone de **M máquinas**, identificadas desde la máquina 0 hasta la máquina M−1.

Cada máquina puede ejecutar uno o varios procesos consecutivos, pero la fabricación debe respetar el orden establecido: una vez que una máquina termina su bloque de procesos, la siguiente máquina ejecuta el bloque posterior. Cada máquina debe tener asignado al menos un proceso y todos los procesos deben ejecutarse exactamente una vez.

Sin embargo, no todas las máquinas pueden ejecutar todos los procesos. Además, se conoce el costo de ejecutar cada proceso en cada máquina, el cual puede variar según las características técnicas y el consumo energético de cada equipo.

Se desea determinar una asignación de los procesos a las máquinas que **minimice el costo total de fabricación**, respetando el orden de los procesos y las restricciones de compatibilidad. Si no existe una asignación que cumpla todas las condiciones, se deberá informar que no es posible completar la fabricación bajo las restricciones establecidas. A continuación, algunos ejemplos:

Caso 1:

Considere (N=4) procesos y (M=2) máquinas.

Matriz de costos

Proceso	Máquina 1	Máquina 2
1	3	6
2	4	2
3	5	3
4	6	4

Matriz de compatibilidad

Proceso	Máquina 1	Máquina 2
1	1	1
2	1	0
3	0	1
4	1	1

Una asignación válida consiste en ejecutar los procesos 1 y 2 en la máquina 1, y los procesos 3 y 4 en la máquina 2. El costo total es: 3+4+3+4=14

Salida esperada:

Costo mínimo total: 14

Caso 2:

Considere (N=3) procesos y (M=2) máquinas.

Matriz de costos

Proceso	Máquina 1	Máquina 2
1	2	5
2	4	3
3	6	2

Matriz de compatibilidad

Proceso	Máquina 1	Máquina 2
1	1	0
2	0	1
3	1	0

En este caso, cada proceso tiene al menos una máquina compatible. Sin embargo, no es posible asignar un bloque no vacío a cada máquina respetando el orden de los procesos y de las máquinas.

Salida esperada:

No existe una asignación.

Para resolver el problema, desarrolle un programa en C++ que utilice **programación dinámica** para encontrar el costo mínimo total de fabricación. Considere las siguientes condiciones:

- Los procesos deben ejecutarse en el orden en que fueron definidos.
- Cada máquina tiene un costo específico para ejecutar cada proceso.
- Cada proceso debe ser ejecutado por una máquina compatible.
- Cada proceso debe asignarse exactamente una vez.
- Los procesos asignados a una misma máquina deben formar un bloque consecutivo.
- Los bloques de procesos deben asignarse respetando el orden de las máquinas.
- Cada máquina debe recibir al menos un proceso.

Para que la solución sea válida se debe mostrar la matriz, si la respuesta emplea recursión o no muestra la matriz de la pregunta queda anulada.

Pregunta 3 (10 puntos)

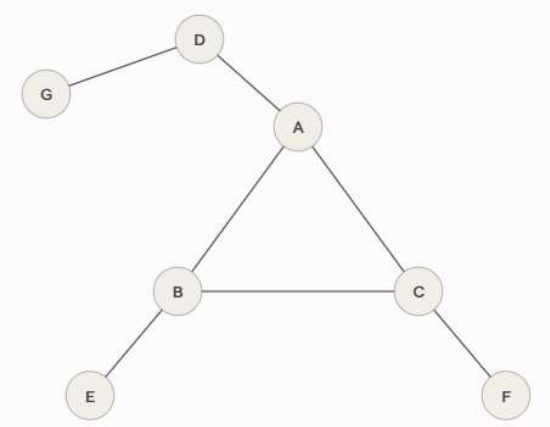
El operador de telecomunicaciones OSCURO S.A.C. debe habilitar un conjunto de antenas distribuidas en Lima. El problema surge con el concepto de la interferencia: dos **antenas cuyas áreas de cobertura se solapan (se chancan) o que están lo bastante cerca no pueden transmitir en el mismo canal de frecuencia sin degradar la señal, de modo que a cada par de antenas conflictivas se le exige operar en canales distintos**.

El espectro radioeléctrico es un recurso escaso y costoso —cada canal adicional que la empresa habilita implica licencias, equipamiento y gestión—, por lo que interesa **cubrir todas las antenas utilizando la menor cantidad posible de canales diferentes**.

La información que maneja OSCURO para la ubicación de las antenas es la siguiente:

- Para cada antena, el conjunto de antenas con las que entra en conflicto, lo que se denomina su "mapa de interferencias".
- Se trabajan en grafos con información adicional siendo cada nodo donde se ubicará la antena. Nodos consecutivos a un nodo donde hay una antena, constituye su mapa de interferencias.
- Se recorre cada nodo donde haya una antena y se le asigna a ésta, el canal de menor índice que no esté ya en uso por ninguna de las antenas conflictivas previamente atendidas.
- Si todos los canales existentes generan conflicto, se habilita un canal nuevo.
- Una asignación es viable mientras no existan dos antenas en conflicto compartiendo canal.
- El conjunto solución S es el mapeo final de antenas ubicadas con sus respectivos canales

Véase el siguiente grafo:



Considere siete antenas a colocar, de A a G, con estos conflictos: A interfiere con B, C y D; B con A, C y E; C con A, B y F; D con A y G; E con B; F con C; G con D.

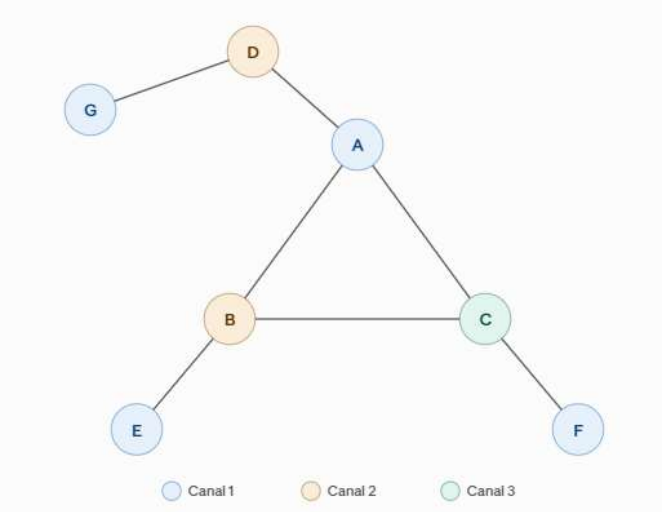
OSCURO empieza por la antena que está en A y le asigna el canal 1. Sigue con la antena en B, que interfiere con A, así que no puede usar el canal 1 y recibe el canal 2 y así sucesivamente, **minimizando el uso de canales.**

Resultado:

Antena A -> Canal 1
Antena B -> Canal 2
Antena C -> Canal 3
Antena D -> Canal 2
Antena E -> Canal 1
Antena F -> Canal 1
Antena G -> Canal 1

Se pide que desarrolle un **algoritmo voraz** que ayude a la empresa a colocar las antenas en los nodos y a asignar los canales, buscando minimizar la cantidad de canales. El programa:

a) Deberá usar matrices de adyacencia para identificar antenas y mapa de interferencia
b) Deberá indicarse **claramente el criterio voraz seguido, el cual consiste en un ordenamiento.**
c) El algoritmo debe ser completamente voraz. Para esta pregunta **no está permitido usar ni fuerza bruta ni ningún otro algoritmo o técnica diferente a la voraz vista en clases.**
d) El algoritmo debe mostrar al final las antenas y los canales que estableció para cada una. Del ejemplo anterior:



Al finalizar del examen, comprima cada proyecto por separado empleando el programa Zip que viene por defecto en el Windows, **no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip, o que no cumplan con el nombre indicado.** Luego súbalo a la tarea programa en Paideia para esta evaluación.

Profesores del curso:

Manuel Tupia
Rony Cueva
José Luis Corcuera

San Miguel, 13 de octubre del 2026